

Score Breakdown (10-Point Scale)

Section I (Multiple-Choice): 12 items $\times$ 0.5 pts = **6.0 pts** (60% Easy/Basic)

Section II (Applied Coding): 3 problems = **4.0 pts** (40% Medium)

Total Exam Score: 10.0 points (100%)

NATIONAL ECONOMICS UNIVERSITY

FACULTY OF DATA SCIENCE & AI

OFFICIAL SOLUTIONS & GRADING GUIDE

Course: Data Analysis with Python (DSAI1005)

Content: NumPy (20%), Pandas (20%), Viz (20%), Code (40%)

Difficulty: 60% Easy/Basic + 40% Medium — **Code:** 301-EN

SECTION I: MULTIPLE-CHOICE KEY & RATIONALE (6.0 pts — 0.5 pt each)

(60% Easy & Basic Questions: Foundational Knowledge & Core Syntax Verification)

Question	1	2	3	4	5	6	7	8	9	10	11	12
Topic	NumPy (20%)				Pandas (20%)				Visualization (20%)			
Key	A	B	A	A	B	A	B	B	A	B	A	B

Technical Rationales for Multiple-Choice Items

- **Q1 (Key A):** `np.linspace(0, 10, 5)` generates 5 evenly spaced float values: `[0.0, 2.5, 5.0, 7.5, 10.0]`. `np.arange(1, 6)` generates `[1, 2, 3, 4, 5]`. Multiplying by 2 yields `[2, 4, 6, 8, 10]`. Element-wise addition gives `[2.0, 6.5, 11.0, 15.5, 20.0]`.
- **Q2 (Key B):** Slicing creates an ndarray *view*, sharing the data buffer with matrix `m`. The slice `m[:, -1, 1]` extracts column index 1 in reverse order: element at row 2 (80), row 1 (50), and row 0 (20). Modifying `v[0] = 999` mutates `m[2, 1]` to 999. `v[2]` refers to `m[0, 1]`, which is 20. Output: 999 20.
- **Q3 (Key A):** Arrays align from the trailing dimension: `A : (4, 1, 6)`, `B : (3, 1)`. Adding leading singleton: `B : (1, 3, 1)`. Comparing dimensions: axis 2: `(6, 1) → 6`; axis 1: `(1, 3) → 3`; axis 0: `(4, 1) → 4`. Result shape of `A × B` is `(4, 3, 6)`. Adding `np.zeros(6)` preserves shape `(4, 3, 6)`.
- **Q4 (Key A):** Boolean condition evaluates whether elements are even and strictly greater than 10. Row 0 of `x`: 12 is even and `> 10 → 12 × 2 = 24`; 25 is odd `→ 0`; 8 is not `> 10 → 0`. Row 1 of `x`: 30 is even and `> 10 → 60`; 15 is odd `→ 0`; 22 is even and `> 10 → 44`. Sum along `axis=0` (columns): `[24+60, 0+0, 0+44] = [84, 0, 44]`.
- **Q5 (Key B):** Series addition aligns on index labels. `fill_value=0` replaces missing values before addition. 'Da Nang' is present only in S_1 (`30.0 + 0 = 30.0`); 'Can Tho' is present only in S_2 (`0 + 15.0 = 15.0`).
- **Q6 (Key A):** Label-based slicing (`.loc`) is inclusive of both start and stop boundaries ('Q2', 'Q3', 'Q4' $\rightarrow$ 3 rows). Integer positional slicing (`.iloc`) uses standard Python half-open semantics `[1, 3)` ($\rightarrow$ 2 rows: indices 1 and 2).
- **Q7 (Key B):** Filtering the grouped aggregation table for tuple key ('North', 'Food'): rows with `Region == 'North'` and `Category == 'Food'` are order 0 (Sales 100) and order 4 (Sales 200). Sum = `100 + 200 = 300`.
- **Q8 (Key B):** A Left Merge preserves all records of the left DataFrame `A` ($N = 500$ rows). The 30 rows that have no matching key in table `B` retain their left attributes and receive `NaN` for columns originating from `B`.

- **Q9 (Key A):** In Matplotlib's 2D subplots array, indexing uses 0-based matrix notation `[row, column]`. First row is index 0; second column is index 1. Method call: `axes[0, 1].set_title(...)`.
- **Q10 (Key B):** A Boxplot simultaneously displays the median (50th percentile), interquartile range ($IQR = Q_3 - Q_1$), and flags extreme outliers beyond $1.5 \times IQR$ whisker fences as individual points.
- **Q11 (Key A):** In Seaborn relational plotting, `hue` maps categorical groupings to distinct colors, while `style` maps categories to distinct point marker glyphs (circles, crosses, diamonds).
- **Q12 (Key B):** In `sns.heatmap`, `annot=True` renders correlation numbers inside cells; `plt.tight_layout()` adjusts subplot margins to prevent clipping.

SECTION II: APPLIED CODING SOLUTIONS & RUBRIC (4.0 pts)

(40% Medium-Level Integrated Problems: Vectorized Pipelines, Tabular Analysis & Visual Analytics)

Question 1: Vectorized Computations & Tabular Aggregation (2.0 pts)

Standard Model Solution

```
# a) Vectorized Net Revenue (0.4 pts):
net_revenue = (tx_data[:, 0] * tx_data[:, 1] * (1 - tx_data[:, 2])) - tx_data[:, 3]

# b) Conditional Order Tier Categorization (0.4 pts):
order_tier = np.where((net_revenue >= 250000) | (tx_data[:, 1] >= 4), 'Wholesale', 'Retail')

# c) Left Merge with Customer Profiles (0.4 pts):
merged = orders_df.merge(cust_df, on='cust_id', how='left')

# d) Multi-Level Groupby and Aggregation (0.4 pts):
summary = merged.groupby(['city', 'tier'])['net_rev'].agg(['sum', 'count'])

# e) Descending Sort by Total Net Revenue (0.4 pts):
final_sorted = summary.sort_values(by=('net_rev', 'sum'), ascending=False)
```

Grading Criteria & Partial Credit for Question 1

- **Part a (0.4 pts):** Award 0.2 pts for accurate 2D slice indexing (`tx_data[:, 0]`, etc.); award 0.2 pts for correct algebraic formula with parentheses. Expanded equivalent formula is fully accepted for 0.4 pts.
- **Part b (0.4 pts):** Award 0.2 pts for correct bitwise OR condition with parentheses (`net_revenue >= 250000` | `(tx_data[:, 1] >= 4)`); award 0.2 pts for exact output strings 'Wholesale' and 'Retail'.
- **Part c (0.4 pts):** Award 0.2 pts for join key `on='cust_id'`; award 0.2 pts for join type `how='left'`.
- **Part d (0.4 pts):** Award 0.2 pts for grouping keys `['city', 'tier']`; award 0.2 pts for column selection and `.agg(['sum', 'count'])`. Equivalent dictionary aggregation is awarded full 0.4 pts.
- **Part e (0.4 pts):** Award 0.2 pts for tuple `( 'net_rev', 'sum' )`; award 0.2 pts for `ascending=False`.

Question 2: Multi-Panel Visual Analytics & Outlier Detection (1.5 pts)

Standard Model Solution

```
# a) Matplotlib OO Canvas Initialization (0.5 pts):
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# b) Seaborn Stratified Grouped Boxplot (1.0 pts):
sns.boxplot(
    data=df_sales,
    x='payment',
    y='amount',
    hue='is_fraud',
    palette='Set2',
    ax=axes[0]
)
```

Grading Criteria & Partial Credit for Question 2

- **Part a (0.5 pts):** Award 0.25 pts for grid dimensions (1 row, 2 cols: 1, 2); award 0.25 pts for `figsize=(14, 5)`.
- **Part b (1.0 pts):**
 - **0.3 pts:** Correct data binding and axis variables: `data=df_sales`, `x='payment'`, `y='amount'`.
 - **0.3 pts:** Correct stratification and color palette: `hue='is_fraud'`, `palette='Set2'`.
 - **0.2 pts:** Explicit target axis binding: `ax=axes[0]`.
 - **0.2 pts:** Stated analytical purpose of `hue='is_fraud'`: partitions and visually compares transaction distributions of legitimate vs fraudulent payments side-by-side.

Question 3: Time-Series Revenue Trajectory & Executive Decision-Making (0.5 pts)**Standard Model Solution**

```
# a) 4-Week Rolling Moving Average & Padding (0.3 pts):  
rolling_trend = weekly_sales.rolling(window=4).mean() # or: .rolling(4).mean()  
plt.tight_layout() # command to prevent label clipping
```

Part b: Model Takeaways & Policy Recommendations (0.2 pts):

- **Takeaway 1 (0.1 pt):** Transactions lying above the upper whisker ($Q_3 + 1.5 \times IQR$) are statistical outliers. The boxplot clearly highlights extreme fraud anomalies in Credit Card transactions requiring immediate automated block rules.
- **Takeaway 2 (0.1 pt):** A 4-week rolling moving average filters out short-term weekly noise and random demand spikes, exposing the underlying trend necessary for reliable quarterly budgeting and forecasting.

— END OF SOLUTIONS & GRADING GUIDE —